Structures de données

3ème partie

1. Préambule

1.1 Préambule

Il est fort probable que vous ayez déjà vu ce qu'est une pile ou une file.

Dans ce cours, **nous ne chercherons pas à apprendre comment les implémenter**. L'idée est d'apprendre :

- comment elles fonctionnent
- comment retrouver leur implémentation
- comment les utiliser.

Les langages de référence utilisés dans ce cours sont **Python** et **C++**.

2. Pile (*Stack*)

Definition 2.1.1 (Pile)

Une **pile** (*Stack*) est une structure de données qui a 2 opérations principales :

- *push* qui ajoute un élément dans la pile
- *pop* qui enlève l'élément ajouté le plus récemment dans la pile

Note

Il faut voir une pile de la même façon que vous voyez une pile d'assiettes !

Nous pouvons simuler son fonctionnement facilement avec des listes.

L'idée est la suivante : il suffit de manipuler une liste comme suit :

Opération de pile	**Opération sur une liste**	
	Python	**C++**
push	`list.append()`	`vector.push_back()`
pop	`list.pop()`	`vector.pop_back()`

Attention

Il s'agit de la façon la plus rapide d'utiliser une pile, n'essayez pas d'utiliser des modules spécifiques, on peut perdre de l'information en faisant cela !

2.3 Cas pratique

Une utilisation concrète d'une pile est lorsqu'on cherche à stocker un historique de ce qu'on a vu jusqu'à un instant t d'un algorithme, de manière **ordonnée**.

Par exemple :

- Stocker l'état de récursion dans un programme.
- Retrouver, à l'indice i dans une liste, le premier élément à gauche strictement inférieur à l'élément d'indice i.

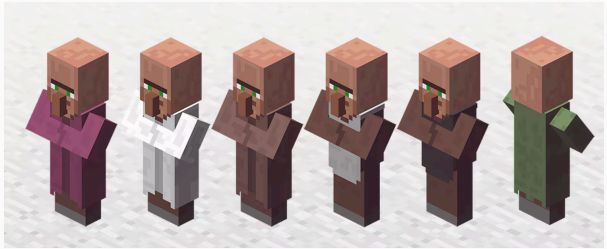
et bien d'autres endroits…

3. File (*Queue*)

Note

Avant de s'attaquer au concept, il est important de s'imaginer une file comme… une file indienne.

Tête de file →



Arrière ←

Definition 3.1.1 (File)

Une **file** (*Queue*) est une structure de données qui a 2 opérations principales :

- *enqueue* qui ajoute un élément à l'arrière de la file
- *dequeue* qui enlève l'élément en tête de la file

3.2 Utilisation

Pour cette structure de données, nous avons accès à des implémentations natives.

Sans perte de généralité, et pour plus de flexibilité, nous utiliserons **exclusivement** ce qu'on appelle des **files à 2 extrémités** (*deque*).

L'intérêt ici est que l'on peut ajouter et supprimer des éléments des 2 côtés de la file.

Pour les utiliser :

- En Python, on a `deque` disponible dans le module `collections`.
- En C++, on a `std::deque` disponible dans le header `<deque>`.

> **Attention**
>
> Un accès dans une *deque* (et une file...) est en $\mathcal{O}(n)$, car la structure maintient une liste chaînée.

3.3 Cas pratique

De manière générale, une file permet de gérer un ordre d'attente ou un flux de traitement.

Nous la retrouvons dans :

- Un *Breadth-First Search* pour que les nœuds soient explorés couche par couche dans un graphe.
- Un système de planificateur de tâches : quelle sera la prochaine tâche à lancer ?

et bien d'autres endroits...

3.4 Gestion de priorité

On est souvent amené à choisir un ordre de priorité que l'on veut.

Pour définir cet ordre, on retrouve 2 fonctionnements différents entre Python et C++.

En C++, c'est simple, `std::priority_queue` demande une fonction de comparaison entre 2 éléments dans son template.

3.5 Cas de Python

En Python, le niveau de flexibilité est faible, mais suffisant pour tous les problèmes.

Pour ce faire, on va utiliser un **tas** (*heap*) qui est une structure qui va maintenir l'élément le plus petit à la racine d'un arbre.

Dans le module `heapq`, on retrouve des outils pour manipuler un tas ; la particularité est qu'il ne propose pas d'objets manipulables avec des méthodes.

Voici comment on l'utilise :

```
from heapq import *
heap = [1, 7, 10, -17]
heapify(heap) # Transforme la liste en tas
print(heappop(heap)) # Enlève -17 du tas et l'affiche
heappush(heap, 6) # Ajoute 6 au tas
```

Quand on exécute `heapify(heap)`, la fonction va ordonner à sa manière les éléments dans `heap` pour le manipuler comme un tas.

L'information est donc présente dans `heap`.

4. Crédits

Auteurs : Bilel MAHDJOUB

Images : Mojang

Pour le **Bureau des technologies** de **Polytech Nice Sophia**